

COMP2113 Operating Systems | Week 7

Deadlock and Starvation

Why does your system freeze when two processes each hold what the other needs?

Roadmap: What You Actually Need

Today we focus on following points:

1. What is deadlock & Starvation?

Definition + four conditions

2. Three ways to handle it:

Prevention / Avoidance / Detection

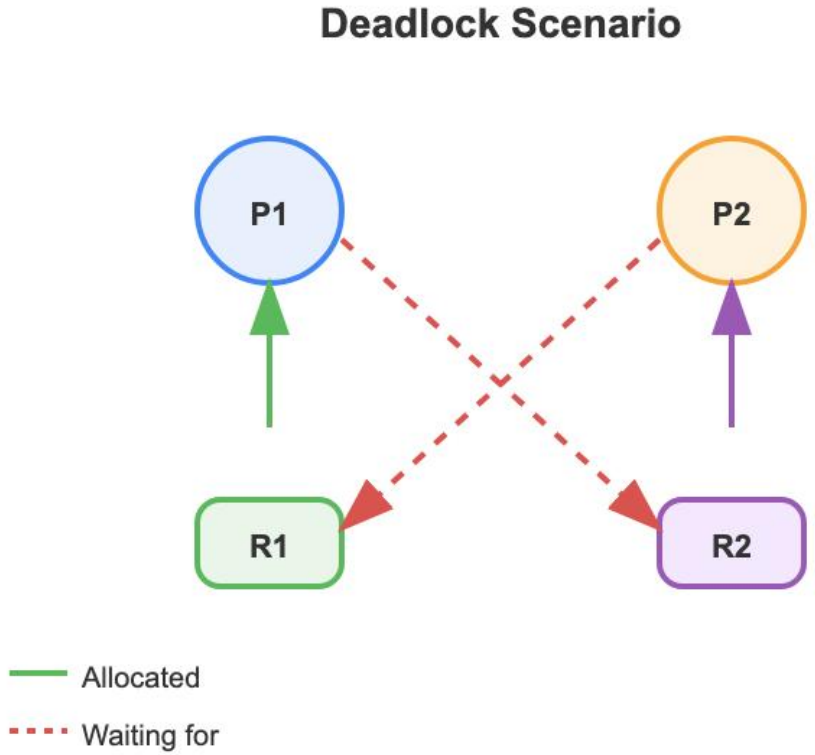
3. Banker's Algorithm (calculation)

Need matrix, Available, safe sequence

4. Compare & tips

What Is Deadlock?

Definition: Deadlock is the permanent blocking of a set of processes, where each process waits for an event (typically release of a resource) that can only be triggered by **another blocked process in the same set.**



Why it is permanent

A deadlock is a condition where no process can proceed for execution because each process is waiting for resources that have been acquired by other processes in the same deadlock set. In this situation, all involved processes become permanently blocked.

Unlike busy-wait (wasting CPU) or normal blocking (will be woken up), deadlock is permanent.

The Four Conditions for Deadlock

Coffman's conditions: all four must hold at the same time for deadlock to occur.

1. Mutual Exclusion

Resource held in mode "only one process at a time".

2. Hold and Wait

Process holds resources already granted while waiting for more.

3. No Preemption

Resources cannot be taken away forcibly; process must release them voluntarily.

4. Circular Wait

Closed chain: each process holds a resource the next process in the chain requests.

Important distinction (often tested)

Conditions 1-3 are **policy choices** we can relax. Condition 4 is a **runtime state** that may or may not emerge. Removing 1-3 removes the *possibility* of deadlock; removing 4 breaks a deadlock that has already formed.

What is Starvation?

Definition: A process waits indefinitely for resources, repeatedly preempted by higher-priority processes; no circular dependency required.

- Root cause：Unfair resource allocation policy (e.g., priority scheduling).
- Solution：Aging: slowly raise priority of waiting processes over time.
- Key distinction from deadlock：Deadlock needs circular wait; starvation does not require circular dependency.

•**Scope** – Deadlock affects multiple processes simultaneously, while starvation typically affects individual processes.
•**Cause** – Deadlock results from circular resource dependencies; starvation results from unfair scheduling policies.
•**Recovery** – Deadlock requires breaking the circular wait or resource preemption; starvation is resolved by fair scheduling mechanisms.

Part 2: Deadlock Prevention

Design the system so that at least one of the four conditions can never hold.

Breaking Each Condition: What We Can Do, and What It Costs

Condition	How to break it	Cost / downside
Mutual Exclusion	Usually **cannot** be broken. If a resource needs exclusive access, the OS must support it. (Readers-writers already relax this for reads.)	We live with it. Deadlock prevention starts with the other three conditions.
Hold and Wait	Require a process to request **all its resources at once**. Block until all are granted. No holding one while waiting for another.	Inefficient: resources held but unused; process may not know future needs; modular code hard to declare all resources upfront.
No Preemption	If a process is denied a new request, it must **release its current resources** and request them again later. Or OS preempts and rolls back.	Only practical for resources whose state can be saved/restored (CPU, memory). For files/databases, needs rollback/recovery.
Circular Wait	Impose a **linear ordering** on resource types. A process may only request resources ordered after the ones it already holds.	Slows processes down; order must be chosen carefully; may be more restrictive than necessary.

Part 3: Deadlock Avoidance

Allow the four conditions, but dynamically refuse a request if it would lead to an unsafe state.

Prevention vs Avoidance: Easy to Mix Up

Aspect	Deadlock Prevention	Deadlock Avoidance
Strategy	Static policy. Remove one of the four conditions at design time.	Dynamic decision. Grant a request only if resulting state is safe.
Future knowledge	Not needed — just forbid patterns.	**Needed.** Each process must declare its maximum needs upfront (Claim matrix).
Concurrency	Low. Restrictive: all-at-once requests, global ordering, forced releases.	Higher. Allows more concurrency as long as state stays safe.
When it acts	At design time / compile time.	At runtime, on every resource request.

Trap: "We always break one of the four conditions." — False. Avoidance keeps all four; it refuses requests that would enter an unsafe state.

Safe vs Unsafe vs Deadlocked State

Safe State

There exists **at least one ordering** in which all processes can run to completion.

The banker's algorithm always keeps the system in a safe state.

No deadlock, regardless of future request order.

Unsafe State

No ordering guarantees all processes finish. Some request sequence may cause deadlock.

Not deadlocked yet — processes might get lucky and release resources.

Banker's algorithm refuses to enter this state.

Deadlocked State

A cycle exists. Some processes **cannot proceed** no matter what order requests come in.

Deadlock has actually happened.

Requires recovery (abort, rollback, preempt).

The Banker's Algorithm: Data Structures & Safety Test

```
// System state
Available[j] : units of resource j free
Allocation[i][j] : held by process i
Max[i][j] : process i's max claim
Need[i][j] = Max - Allocation

// On request req from P:
1. if req > Need: error
2. if req > Available: wait
3. pretend to allocate
4. if safe(new state): grant
else: rollback & wait
```

The safety test

1. Work = Available; Finish[all] = false
2. Find unfinished P where Need[P] ≤ Work
3. If found: Work += Allocation[P]; Finish[P]=true; repeat 2
4. If all Finish=true: **safe**. Else: unsafe.

Intuition

We ask: "if we pretend to grant this request, can we still find some order to run every process to completion, each one releasing its resources to the pool?"

If yes — grant. If no — the request is dangerous, block the process.

The algorithm is called the banker's algorithm because it mirrors a banker deciding whether to lend money while keeping enough cash on hand for all customers.

Banker's Algorithm: Every Variable You Need to Know

Name	What it means	How to get it	Key point
Max	Maximum number of each resource type the process may ever request in total.	Given in the problem. Copy the table.	The process's declared upper limit.
Allocation	Resources the process currently holds right now.	Given in the problem. Copy the table.	When the process finishes, it gives all of these back.
Need	Resources the process still needs to finish.	Need = Max - Allocation (element-by-element)	Do NOT subtract sums. Compare each resource type separately.
Available	Resources currently free in the system, held by no process.	Given, or: Available = Total - Sum(Allocation)	The starting pool for safety check.
Work	Temporary free-resource vector used during the safety test simulation.	Work = Available initially. After a process finishes: Work = Work + Allocation	Add Allocation back, NOT Need. Simulation only.
Finish	Boolean flag per process: can it complete in the simulation?	Start all = false. Set true when Need ≤ Work.	All true = safe. Any false = unsafe.

Practice Problem: Is This State Safe?

System has 3 processes (P0, P1, P2) and 2 resource types (R1, R2). Total resources: **R1 = 10, R2 = 7**.

Max	R1	R2
P0	6	4
P1	3	3
P2	7	2

Allocation	R1	R2
P0	2	1
P1	1	3
P2	3	1

Step 1: Compute Need = Max - Allocation, and Available = Total - Sum(Allocation)

Total allocated: R1 = 2+1+3 = 6; R2 = 1+3+1 = 5
Available = (10-6, 7-5) = (4, 2)
Need = Max - Allocation: P0 = (4,3); P1 = (2,0); P2 = (4,1)

Step 2: Run the Safety Test

```
// Work starts = Available = (4, 2)

P0: Need (4,3) <= (4,2)?
R1 OK, R2: 3 > 2 -- NO
P1: Need (2,0) <= (4,2)? YES
Run P1. Work = (4,2) + (1,3) = (5,5)
P0: Need (4,3) <= (5,5)? YES
Run P0. Work = (5,5) + (2,1) = (7,6)
P2: Need (4,1) <= (7,6)? YES
Run P2. All done.
```

Safe sequence found

P1 → P0 → P2

Conclusion: the state is SAFE.
Any request that preserves this property can be granted.

Tips

1. **Check every resource type separately.** P0 looked like it fit (R1=4 matches) but failed on R2. Compare element-by-element, not by sum.
2. **When a process finishes, add its Allocation back to Work** (not its Need, not its Max).
3. **Write down the sequence.** Even if the state is safe, you must name the order to get full marks.

Why Banker's Algorithm Is Hard in Practice

The theory is clean. Why do real OSes almost never deploy it?

Difficulty 1: No Max declarations

Banker's requires every process to **declare its maximum future resource need in advance**.

In real systems, users and apps do not know this. A browser may open 1 or 100 tabs; a compiler may allocate more memory on large inputs.

Result: the input to the algorithm does not exist.

Difficulty 2: Too slow + unpredictable

Running the safety test on every request is **O(n x m)** where n = processes, m = resource types. With hundreds of processes, this is expensive.

Also, processes may wait arbitrarily long for a "safe" ordering — poor response time.

Result: low throughput, unpredictable latency.

When Banker's Fails: What to Do Instead

If processes cannot declare Max claims, choose a simpler strategy.

Scheme	How it works	Trade-off
Resource ordering	Assign every resource a number; processes must request them in increasing order. Breaks circular wait.	Simple, low overhead. But restricts legitimate request order; may hold resources longer than needed.
Detection + kill	Grant freely. Run detection periodically. If deadlock found, kill one or more processes and restart them.	High concurrency. But killing processes wastes work; detection overhead; deadlock actually happens.
All-at-once	A process must request all resources at once before starting. Breaks hold-and-wait.	Very simple. But low resource utilization; processes may hold resources they don't yet need.

Answer: use resource ordering (prevention) OR detection-and-kill. State the trade-off explicitly — prevention costs concurrency, detection costs wasted work.

Part 4: Detection & Recovery

Grant requests freely. Periodically check whether deadlock has actually occurred, then recover.

Detection Algorithm: Who Is Deadlocked?

Same idea as the safety test, but uses the **current Request matrix Q** (what processes want now), not Max.

```
// Detection
1. Mark all processes unmarked.
2. Mark processes with Allocation row = 0
(they hold nothing, can't deadlock).
3. W = Available.
4. Find unmarked i where
Q[i] <= W.
5. If found: mark i;
W = W + Allocation[i]; goto 4.
6. If not found: stop.

// Unmarked processes are deadlocked.
```

Recovery strategies

1. **Abort all deadlocked processes.** Simplest; also most wasteful.
2. **Rollback to checkpoint.** Restart from a saved state; deadlock may recur.
3. **Abort one at a time**, cheapest process first, re-run detection.
4. **Preempt resources** one at a time; roll back victim.

Critical distinctions

Banker's (Avoidance): We try NOT to get into trouble. We never let the system enter an unsafe state. Deadlock may never arise. Needs advance Max claim from every process.

Deadlock-Detection: We get into trouble, then find & fix it. All resource requests are granted. We check later: "are some processes already stuck in deadlock?" No need for max resource declaration. If deadlock exists, perform recovery actions.

Important terminology reminder:

- Prevention, **Avoidance**, **Detection-Recovery** are three distinct major families for deadlock handling.
- Banker's algorithm is the representative algorithm for Avoidance.
- Deadlock-detection algorithm belongs to Detection-Recovery.

Three Strategies Compared + Integrated Approach

Aspect	Prevention	Avoidance	Detection
When	Design time	Runtime, each request	Periodically / on demand
Future know.	Not needed	Required (Max claim)	Not needed
Overhead	Low at runtime; restrictive	High: run safety check	Detection cost + recovery cost
Concurrency	Low	High	Highest (no restriction)

***Integrated strategy (Howard, 1973)**

Real OSes mix strategies by resource class: impose resource ordering *between* classes; inside each class, use the best fit — prevention for swappable space, avoidance for process resources, preemption for main memory, ordering for internal resources.

Banker's Algorithm Practice Question

Resources: 3 resource types: R0, R1, R2 Total
system resources: Total = ([12, 8, 10)
There are 4 processes: P0, P1, P2, P3

Process	Max (maximum demand)	Allocation (currently held)
P0	[7, 5, 4]	[2, 1, 2]
P1	[5, 3, 3]	[3, 1, 0]
P2	[9, 4, 5]	[2, 2, 2]
P3	[4, 4, 4]	[1, 1, 3]

Available (free resources in system): Available = [4, 3, 3]

Tasks

1.Compute the Need matrix for all processes.
2.Run safety-check algorithm. Is the system in a safe state? If yes, give one valid safe sequence.

Key Takeaways

1. **Deadlock** = permanent blocking; four necessary conditions: mutual exclusion, hold-and-wait, no preemption, circular wait.
2. **Prevention** breaks one condition by design. Strength: simple at runtime. Weakness: low concurrency.
3. **Avoidance (Banker's)** dynamically stays in a safe state. Strength: high concurrency. Weakness: needs Max declarations, O(nm) per request.
4. **Detection + recovery** lets deadlock happen, then finds and kills. Strength: no restriction. Weakness: wasted work, recovery cost.
5. **Calculation steps:** Need = Max - Allocation; Available = Total - sum(Allocation); repeatedly find a process whose Need ≤ Work, run it, add its Allocation to Work.
6. **Safe ≠ deadlocked**, but **unsafe ≠ deadlocked yet**. Safe sequence proves no deadlock can occur.

Practice: Quick Check

True or false?

Q1. Need[i][j] = Max[i][j] + Allocation[i][j].

Q2. An unsafe state is necessarily a deadlocked state.

Q3. The Banker's algorithm requires each process to declare its maximum resource need.

Q4. When a process finishes in the safety test, we add its Allocation back to Work.

Q5. Deadlock prevention is generally less restrictive than deadlock avoidance.

Q6. Resource ordering breaks the circular-wait condition.

Q7. Detection-and-recovery never wastes completed work.